



NO CAP

Technical Requirements Document

Design it. Break it. Scale it. — for one learner, at zero cost.

DOCUMENT	Technical Requirements Document
VERSION	v2.1 (personal app, zero-cost)
DATE	16 August 2026
ARCHITECTURE	Cloudflare-native (Workers + D1 + R2 + DO + KV)
COST TARGET	Rs 0/month forever, no trial credits, no credit card
STATUS	Working spec

PRODUCT THESIS

A personal system-design workbench that turns passive reading into a repeatable loop of learning, visualization, reasoning, simulation, architecture practice, and interview readiness — deployable for zero cost, forever, on Cloudflare's genuine free tier, with hard quota guards instead of accidental billing.



What Changed: v2.0 to v2.1

v2.0 proposed a production-grade architecture (managed PostgreSQL, Redis, Celery, S3, paid TTS/STT providers, live cloud pricing APIs, Firecracker code execution). v2.1 rewrites the entire infrastructure layer for one requirement: **zero monthly cost, forever, with no trial credits and no accidental billing**. The frontend and domain model are largely unchanged; the backend is Cloudflare-native.

Infrastructure Pivots

- PostgreSQL → **Cloudflare D1** (SQLite, 5GB free, 5M rows read/day, 100k rows written/day, fail-closed).
- Redis → **removed**. Caching via Cloudflare KV; presence via Durable Objects; job coord via Queues.
- Celery/RQ → **Cloudflare Queues + Cron Triggers** (10k operations/day free, 24h retention).
- S3 → **Cloudflare R2** (10GB free, 1M Class A + 10M Class B operations/month, free egress).
- FastAPI server → **FastAPI on Python Workers** (Pyodide runtime, 100k requests/day free). Pure-Python domain logic only.
- WebSocket + Yjs server → **Durable Objects** + WebSockets (100k requests/day, 13k GB-s/day free, SQLite-backed).
- TTS/STT providers → **Web Speech API** (browser-native, zero cost). External providers are Tier D.
- Cloud pricing APIs → **bundled pricing dataset** (versioned JSON in git, refreshed quarterly via PR).
- Firecracker/e2b code execution → **deferred**. Code Walkthrough Mode replaces it (read + modify + predict, no execution).
- AI mentor providers → **Cloudflare Workers AI** (10k neurons/day free, quota-guarded, deterministic fallback).

Architecture Stance

- Modular monolith stays. Single Worker, single D1, single R2.
- Static content bundled at deploy time (versioned JSON/MDX in git).
- Dynamic data (user state only) in D1.
- Every external dependency has a deterministic fallback.
- Quota guards fail-closed: feature unavailable, not billed.
- Quota dashboard visible in UI (Tier A).
- No microservices, no Kubernetes, no event streaming, no vector DB.
- PWA service worker for offline core shell + lesson packs.
- GitHub OAuth (single user). No password storage.
- Cloudflare Pages for static hosting + Workers for API.

THE HONEST TRUTH ABOUT FREE TIERS

Cloudflare's free tier is genuine and explicitly "fail-closed" (queries fail rather than billing you), but it is not infinite. 100k Worker requests/day, 5M D1 rows read/day, 10k Workers AI neurons/day — these are real numbers. The correct architecture is "free by construction, never accidentally billable", not "pretend free tiers are infinite". Every optional service has a quota guard, a fallback, and a graceful-degradation path baked in from day one.

1. Technical Vision

Use a Cloudflare-native architecture that costs zero dollars per month, forever. The product is not required to be fully offline, but the PWA service worker makes previously loaded content, notes, and saved architectures usable offline. The product remains useful when AI, voice, or enrichment services fail (or when their free quotas are exhausted). Every new capability is wrapped in an adapter with a deterministic fallback — the learner is never blocked by infrastructure they cannot see, and never billed for infrastructure they did not explicitly enable.

2. Recommended Stack

Layer	Choice	Reason / Free Tier
Web frontend	Next.js + TypeScript (static export) on Cloudflare Pages	Static hosting on Pages is free, unlimited. PWA installable.
API	FastAPI on Python Workers (Pyodide runtime)	100k requests/day free, fail-closed. Pure-Python domain logic.
Mobile	PWA (installable, no app store). Native (React Native) is Tier D.	PWA is free. Native requires app store accounts + build infra.
UI	Tailwind CSS + accessible component primitives	No runtime cost. Glassmorphism via native CSS backdrop-filter.
Glassmorphism	Native CSS backdrop-filter + Tailwind utilities	No glass library - native CSS handles blur, saturate, inset shadows.
Primary DB	Cloudflare D1 (SQLite)	5GB storage, 5M rows read/day, 100k rows written/day. Fail-closed.
Object storage	Cloudflare R2 (S3-compatible)	10GB storage, 1M Class A + 10M Class B operations/month, free egress.
Cache / ephemeral	Cloudflare KV (OPTIONAL enrichment cache, not core)	1GB storage, 100k reads/day, 1k writes/day free. Use only for link previews and AI prompt cache. Core app works without it. Browser cache + Cloudflare edge cache + D1 cover the common paths.
Realtime	Cloudflare Durable Objects + WebSockets	100k requests/day, 13k GB-s/day free. SQLite-backed. For collab presence + CRDT sync.
Background jobs	Cloudflare Queues + Cron Triggers	10k operations/day free, 24h retention. For review scheduling, weekly digest.
Search	D1 FTS5 (SQLite full-text search)	Free, built into D1. Phase 2: typo tolerance if needed.

Layer	Choice	Reason / Free Tier
Diagram editor	React Flow / equivalent graph editor	Client-side. Zero server cost. Saves as JSON to D1.
Charts / visualization	SVG / Canvas + a small chart library	Client-side. Zero server cost.
Code execution	DEFERRED. Code Walkthrough Mode replaces it (read + modify + predict).	No infrastructure. Live execution is Tier D.
TTS	Web Speech API (browser-native SpeechSynthesis)	Zero server cost. Voice quality varies by browser/OS. External TTS is Tier D.
STT	Web Speech API (browser-native SpeechRecognition)	Zero server cost. Chrome/Edge support is good; Safari partial; Firefox limited.
Cloud pricing	Bundled pricing dataset (versioned JSON in git)	Zero server cost. Refreshed quarterly via PR. ~20 AWS services initially.
Calendar	DEFERRED to Tier C. Manual session reminders in MVP.	No OAuth complexity in MVP. Calendar integration is opt-in later.
AI	Cloudflare Workers AI (quota-guarded, 10k neurons/day free) with model registry	Optional Tier C feature. Model registry selects free-compatible models (some models now require Workers Paid - e.g. kimi-k2.6, glm-5.2). Deterministic fallback when quota exhausted.
Auth	AuthProvider abstraction (GitHub now, Google future) via Worker	Free. No password storage. Single-user app = no multi-tenant complexity. Lightweight provider interface so adding Google later costs nothing architecturally.
Observability	Cloudflare Workers Analytics + custom quota dashboard	Free built-in analytics. Custom quota dashboard in UI.
CI/CD	GitHub Actions + Cloudflare Pages/Workers deploy	Free for personal repos. Auto-deploy on push to main.

3. High-Level Architecture



v2.1 Cloudflare-native architecture. All components on genuine free tier.

The entire product runs on Cloudflare's free tier. There is no managed PostgreSQL, no Redis, no Celery worker pool, no S3, no Yjs server. Static content is bundled at deploy time. User state lives in D1. Assets in R2. Caching in KV. Realtime in Durable Objects. Background work in Queues + Cron. Optional AI in Workers AI. Every component fails-closed when its quota is hit — no surprise bills.

4. Architectural Principles

- Static-first: content (concepts, lessons, diagrams, quizzes, case studies, pricing data) is versioned JSON/MDX in git, built into static bundles at deploy time. Zero DB queries for content reads.
- D1 is the source of truth for user state only: progress, mastery, review schedule, attempts, notes, architectures, collections.
- Interactive labs execute deterministic client-side simulation logic. Zero Worker requests per lab session.
- AI calls are asynchronous, cancellable, and quota-guarded. Never block core navigation on an AI response. Workers AI free tier: 10k neurons/day.
- Voice/TTS/STT use browser-native Web Speech API. Zero server cost. External providers are Tier D.
- Cost data is bundled (versioned JSON in git, refreshed quarterly). No live API calls. Works offline.
- External resources (link previews, citation previews) are cached in KV with 7d TTL and wrapped by a Worker fetch with 5s timeout.
- Content is versioned so edits do not invalidate historical attempt records.
- Events are recorded for learning analytics, not for every trivial UI interaction. Batched client-side, sent to D1 in groups.
- WebSocket connections for collaboration are stateless on the server — all state lives in the Durable Object + CRDT document synced to R2 on save.
- Quota guards are mandatory on every optional service. Quota status is visible in the UI. Failures degrade gracefully, never bill.

5. Domain Model

v2.1 keeps the v2.0 domain entities (they were right) but resequences them by tier. Tier A entities ship in 0.1/0.5. Tier B entities ship in 1.0. Tier C entities ship in 2.0. The schema is designed so each tier is additive — no backward-incompatible migrations.

Entity	Purpose	Tier
User	Identity (single user in MVP), preferences, timezone, learning settings. GitHub OAuth subject.	A
Track	Curriculum grouping such as System Design Foundations or AWS.	A
Phase	Ordered section of a track.	A
Concept	Atomic learning unit with prerequisites and mastery metadata. Content lives in static bundles; D1 stores only metadata + mastery.	A
ConceptVersion	Immutable published snapshot of content. Tracked in git, not D1.	A
LessonBlock	Typed content block: text, diagram, code, image, callout, interactive, table. Lives in static bundles.	A
QuizQuestion	Assessment item with options, answer, rationale, difficulty. Lives in static bundles.	A
QuizAttempt	User response, score, timing, feedback. Stored in D1.	A
ReviewItem	Spaced repetition state for a concept or question. D1.	A
Simulation	Deterministic lab definition and parameter schema. Static bundle.	A
SimulationAttempt	Inputs, result, score, timestamps. D1.	A
ArchitectureExercise	Challenge definition, constraints, rubric. Static bundle.	A
ArchitectureAttempt	Saved graph (JSON), evaluation, feedback. D1.	A
CaseStudy	Guided design scenario with requirements and reference architecture. Static bundle.	A
Project	Hands-on build brief, milestones, evidence. Static bundle.	B
InterviewSession	Timed session, prompts, responses, rubric results. D1.	A (text) / C (voice)

Entity	Purpose	Tier
Resource	Optional external reference with URL and metadata. KV cache.	C
LearningEvent	Normalized event used for mastery and analytics. D1 (partitioned by month).	A
Achievement	Milestone / badge definition. Static bundle.	A
UserAchievement	Earned achievement record. D1.	A
Note	Personal note attached to a concept. Free-form text, optional links to other concepts. D1.	B
Annotation	Margin annotation anchored to a specific paragraph or diagram element. D1.	B
Highlight	Text selection colored with one of four learner-defined categories. D1.	B
Collection	User-defined study list of concepts. D1.	B
CollectionItem	Concept-in-collection with order_index and added_at. D1.	B
CodePattern	Runnable code snippet for a concept. Static bundle (no execution).	B
CostEstimate	Saved cost calculation. Architecture hash, cloud, region, breakdown JSON, as_of date. D1.	B
CareerPath	Target role + gap analysis snapshot + generated 4-week plan. D1.	B
VoiceSession	TTS narration session metadata. Audio not stored (browser-native TTS). D1.	C
VoiceInterviewAttempt	Voice interview transcript (audio not stored; STT is browser-native). D1.	C
CollabSession	Active collaboration session. Owner, invitees, CRDT document R2 key, started_at, ended_at. D1 + Durable Object.	B (v2.0)
CloudPricing	Bundled pricing dataset. Versioned JSON in git, NOT in D1. ~20 AWS services initially.	B
QuotaUsage	Daily quota usage tracking. Worker requests, D1 rows, R2 ops, AI neurons. D1 (reset by Cron).	A

WORKER CONSTRAINT: THIN PYTHON ONLY

FastAPI on Python Workers runs in Pyodide (WebAssembly). Limits: 10ms CPU time, 128MB memory, 3MB Worker size, 50 subrequests per request. **No heavyweight Python libraries:** no NumPy, no Pandas, no SciPy, no native binary dependencies, no heavyweight ML, no image processing, no server-side simulation engines. The Worker is a thin API/domain layer. All heavy computation (simulations, architecture validation, cost calculation) runs client-side. This is why the static-first + client-side-compute architecture is not just a cost optimization — it is a hard requirement of the runtime.

6. Content Architecture (Static-First)

Content is the biggest insight in v2.1. NO CAP has roughly 300-500 concepts, 50-100 simulations, 30-50 case studies, hundreds of questions, thousands of content blocks. That is tiny. There is no reason to query a database for content. Content lives as versioned JSON/MDX in a git repository, built into static bundles at deploy time, served from Cloudflare Pages edge.

Zero DB queries for content reads.

```
Git repository (content/)
|
+-- concepts/
|   +-- fundamentals/
|       |   +-- how-the-internet-works.json
|       |   +-- dns.json
|       |   +-- ...
|   +-- scaling/
|   +-- databases/
|   +-- ... (16 areas total)
|
+-- simulations/
|   +-- load-balancer.json
|   +-- cache-aside.json
|   +-- ...
|
+-- case-studies/
|   +-- whatsapp.json
|   +-- url-shortener.json
|   +-- ...
|
+-- pricing/
|   +-- aws.json (bundled, refreshed quarterly via PR)
|   +-- gcp.json (later)
|   +-- azure.json (later)
|
+-- code-patterns/
|   +-- cache-aside-python.json
|   +-- cache-aside-go.json
|   +-- ...

Build step (CI/CD):
content/*.json -> Next.js static props -> Pages edge cache

Runtime:
- Content reads: zero Worker requests, zero D1 queries
- User state reads/writes: D1 (small, fast, free)
- Asset reads: R2 (only for images/audio/exports)
```

Static first content architecture. Content in git, served from edge.

This separation is the biggest cost win in v2.1. A learner browsing 50 concepts in a session generates **zero** Worker requests and **zero** D1 reads for the content itself. Only their progress, attempts, and notes hit D1 — and those are small writes/reads. The 5M D1 rows read/day free tier becomes effectively unlimited for a single user.

7. API Strategy

The API is split: **content endpoints are static** (served from Pages, zero Worker cost), **user state endpoints are dynamic** (served from Worker + D1). This keeps Worker requests minimal — only mutations and user-specific reads hit the Worker.

Area	Style	Examples	Tier
Auth	Worker + GitHub OAuth	/auth/login, /auth/callback, /auth/me	A
Content (static)	Next.js static props from JSON	/concepts/[slug], /tracks, /case-studies/[slug]	A
Content (dynamic metadata only)	Worker + D1	/concepts/[slug]/mastery (returns user's mastery for this concept)	A
Progress	Worker + D1 + event batch endpoint	/progress, /events/batch	A
Assessment	Worker + D1	/quizzes/[id]/attempts, /simulations/[slug]/attempts	A
Architecture	Worker + D1	/exercises/[slug]/attempts (save/evaluate)	A
Interview	Worker + D1	/interviews, /interviews/[id]/responses	A (text) / C (voice)
AI	Worker + Workers AI (quota-guarded)	/mentor/sessions, /mentor/jobs/[id]	C
Notes	Worker + D1	/notes, /notes/[id], /concepts/[slug]/notes	B
Annotations	Worker + D1	/annotations, /annotations/batch	B
Highlights	Worker + D1	/highlights, /highlights/batch	B
Collections	Worker + D1	/collections, /collections/[id]/items	B
Code patterns (static)	Next.js static props from JSON	/patterns/[slug] (no execution endpoint)	B
Cost	Worker + bundled pricing JSON	/cost/estimate (POST body: architecture JSON, cloud, region)	B
Career	Worker + D1	/career/path, /career/gap-analysis, /career/plan	B
Voice	Worker + D1 (metadata only; TTS/STT client-side)	/voice/sessions, /voice/interviews/[id]/transcript	C

Area	Style	Examples	Tier
Collab	Durable Object + REST	WS /collab/[session_id], REST /collab/sessions, /collab/invite	B (v2.0)
Quota	Worker + D1	/quota/today (returns today's usage for all services)	A

8. Versioning

Version the API at the contract boundary (URL prefix /v1/), and version content independently via git. Published content versions are immutable git tags. New edits create a new draft PR and then a new published version on merge. Historical quiz attempts, code walkthrough attempts, and cost estimates reference the version they were generated against (stored as a version string in the attempt record). Cost estimates additionally reference the pricing dataset version (as_of date).

9. Authentication and Authorization

- GitHub OAuth only. No email/password. No multi-tenant complexity.
- Single-user app: the OAuth allow-list is one GitHub account (the author). Other accounts are rejected at the Worker.
- Short-lived JWT access token (1h) + refresh token (7d) stored in httpOnly secure cookie.
- No roles system in MVP (single user = admin + learner + editor).
- Notes, annotations, highlights, and personal architectures are private by definition.
- Code Walkthrough has no execution surface — no sandboxing needed.
- Quota guards run on every Worker request: if user is over daily limit, return 429 with retry-after = midnight UTC.
- PWA service worker caches auth state for offline use (re-validates on reconnect).
- Tier C features (AI, voice interview) require explicit opt-in (toggle in Settings). Off by default.
- Tier B (v2.0) collab sessions: invite URLs use 32-byte random slugs. Only the owner can delete or archive.

10. Data and Persistence Requirements

Data	Persistence	Retention	Tier
Core content (concepts, lessons, quizzes, simulations, case studies)	Git repo (versioned JSON/MDX) -> Pages static bundles	Permanent (git history)	
User progress + mastery	D1 (SQLite)	Long-lived	A
Review schedule	D1	Long-lived	A

Data	Persistence	Retention	Tier
Quiz/simulation/architecture attempts	D1	Long-lived (5 years)	A
Interview sessions (text)	D1	Long-lived	A
Telemetry (learning events)	D1, partitioned by month	12 months	A
Media assets (authored diagrams, images)	R2 + Pages edge cache	Permanent	A
Link preview cache	KV (7d TTL)	7 days	C
Notes & annotations	D1	Long-lived, user-deletable	B
Collections	D1	Long-lived, user-deletable	B
Cost estimates	D1 (with as_of date)	Long-lived	B
Career paths	D1	Long-lived	B
Voice session metadata	D1 (audio not stored - browser TTS)	90 days	C
Voice interview transcripts	D1 (text only)	User-configurable (default 30 days)	C
Collab session CRDT docs	R2 (snapshot on save) + Durable Object (live state)	90 days post-session	B (v2.0)
Bundled cloud pricing	Git repo (versioned JSON)	Permanent (git history)	B
Quota usage tracking	D1 (reset daily by Cron)	7 days rolling	A
AI mentor session history	D1	User-deletable	C

11. Interactive Simulation Architecture

All labs are deterministic simulations executed **entirely client-side**. Each simulation defines a state model, parameters, constraints, and output metrics. The frontend executes the simulation locally — zero Worker requests, zero D1 queries, zero cost. v2.1 keeps the v2.0 cost model output but uses the bundled pricing dataset (not live APIs) when the Cost Simulator lab ships in Tier B.

```
SimulationDefinition (static JSON in git)
  id
  version
  inputs:
    name, type, default, min, max
  engine:
    state_transition_function (JavaScript, runs client-side)
  outputs:
    metric definitions
    (latency, throughput, error_rate)
    cost_per_hour [Tier B - uses bundled pricing]
  cost_model [Tier B]:
    instance_type_mapping (e.g., "server_count -> m6i.xlarge")
    region_default
    reserved_default
    pricing_dataset_version (e.g., "aws-2026-08")
  lessons:
    concept refs
  validation:
    expected behaviors

Runtime:
  - All computation in browser via Web Worker (off-main-thread)
  - Zero network requests during simulation
  - Save attempt to D1 only on explicit "Save" or "Submit"
```

Client-side SimulationDefinition. Zero server cost per session.

12. Architecture Evaluation Engine

The evaluator runs entirely client-side. It receives a graph and exercise rubric, then runs deterministic rules such as SPOF detection, illegal edge detection, exposure checks, dependency depth, missing durability, expected-component checks, and (Tier B) cost-annotation validation. AI can later explain the findings in natural language (Tier C, quota-guarded) but cannot mutate the score. **Zero Worker requests per validation.**

13. Search

Tier A MVP: client-side fuzzy search over static content (concepts + glossary). Zero Worker requests. Built with a small in-memory index (FlexSearch or similar) loaded from the static bundles. **Tier B (v1.0):** extends to include personal notes and collections via D1 FTS5 (SQLite full-text search, free, built into D1). Search results group by type with personal content

surfaced above public content when relevance is similar.

14. Background Jobs

Cloudflare Queues + Cron Triggers replace Celery/RQ. Free tier: 10k operations/day, 24h retention. Jobs are idempotent and bounded-retry.

Job	Trigger	Retry	Tier
Review scheduling	Concept completion / nightly Cron reconciliation	Safe retry	A
Analytics aggregation	Event batch threshold / hourly Cron	Idempotent	A
Quota reset (daily)	Cron at midnight UTC	Idempotent	A
Quota warning check	Cron every hour	Idempotent	A
External resource refresh (link previews)	Manual / weekly Cron	Low-frequency retry	C
Export generation (portfolio PDF)	User request via Queue	Bounded retry	B
Weekly digest	Cron Sunday 9pm user-local	Idempotent	B
AI mentor request	User request via Queue	Bounded retry + timeout	C
Pricing dataset refresh check	Monthly Cron (reminds author to PR update)	Idempotent	B
Cost estimate revalidation	On pricing dataset version change	Idempotent	B
Voice interview transcript save	On session end (browser STT done client-side)	Bounded retry	C
Collab session snapshot	Every 60s via Durable Object + on session end	Idempotent	B (v2.0)
Streak recovery token grant	Daily Cron check	Idempotent	A

15. External Online Integrations

v2.1 has far fewer external integrations than v2.0. TTS/STT are browser-native. Cloud pricing is bundled. Code execution is deferred. AI is Workers AI. The only remaining external integration is link/citation preview caching — and even that is Tier C (optional).

Adapter	Mechanism	Timeout	Cache Strategy	Fallback
AI (mentor)	Workers AI (Cloudflare internal)	15s per request	Prompt-hash KV cache (24h)	Authored explanation from content model
TTS	Web Speech API (browser-native)	N/A (client-side)	N/A	Text mode (no audio)
STT	Web Speech API (browser-native)	N/A (client-side)	N/A	Manual text input
Cloud pricing	Bundled JSON (no API call)	N/A (no network)	Permanent (git versioned)	Always available
Link preview	Worker fetch + KV cache	5s	7d TTL in KV	Show URL only, no preview card
GitHub OAuth	Worker fetch to github.com	5s	N/A	Auth fails gracefully; retry
Code execution	DEFERRED (Tier D)	-	-	Code Walkthrough Mode (no execution)

16. Performance Strategy

- **Static content served from Cloudflare Pages edge.** LCP target < 1.8s for Home, < 2.5s for content pages.
- **Content reads generate zero Worker requests.** Next.js static props + edge cache.
- **User state reads hit D1.** Small queries, indexed. Target < 50ms p95.
- **Batch learning events client-side.** Send to D1 in groups of 10-20, or on page unload.
- **Use optimistic UI for progress updates.** Update local state immediately, sync to D1 in background.
- **Keep simulations client-side.** Run in Web Worker (off-main-thread) to avoid UI jank.
- **PWA service worker caches core shell + current lesson + notes.** Offline-capable for read-only use.
- **WebSocket connections use a single Durable Object per collab session.** No fan-out infrastructure.
- **Cost estimate calculation is client-side.** Bundled pricing data is small (~50KB JSON). Calculation < 10ms.
- **Quota dashboard updates client-side.** Polls /quota/today every 5 min; updates instantly on any mutation.
- **Code Walkthrough has no execution.** "Predict output" reveals authored expected output instantly.
- **Search is client-side (Tier A) or D1 FTS5 (Tier B).** No external search service.

17. Security

- TLS everywhere (Cloudflare auto-TLS).
- Secrets via Cloudflare Workers Secrets (encrypted at rest). Never in repo.
- D1 queries use parameterized statements (SQLite prepared statements).
- CSRF protection via SameSite=Strict cookies + custom header check.
- Rate-limit auth endpoints (5 attempts / minute per IP).
- Sanitize user-authored markdown (notes, collection descriptions) via DOMPurify client-side + re-sanitize on read.
- No file uploads in MVP. (Notes are text-only. Code Walkthrough has no execution.)
- Audit privileged content changes via git history (content is in git, not D1).
- Never expose internal API keys to clients. All secrets server-side (Worker Secrets).
- GitHub OAuth: scope limited to read:user (just to identify the user). No repo access.

- Voice interview transcripts (Tier C): stored in D1, deletable. Audio is NOT stored (browser STT transcribes locally).
- Collab sessions (Tier B, v2.0): invite URLs use 32-byte random slugs. Only the owner can delete or archive.
- PWA service worker: cache strategy is "stale-while-revalidate" for content, "network-first" for user state.
- No SOC 2 needed (single-user personal app). Revisit if app ever goes multi-user.

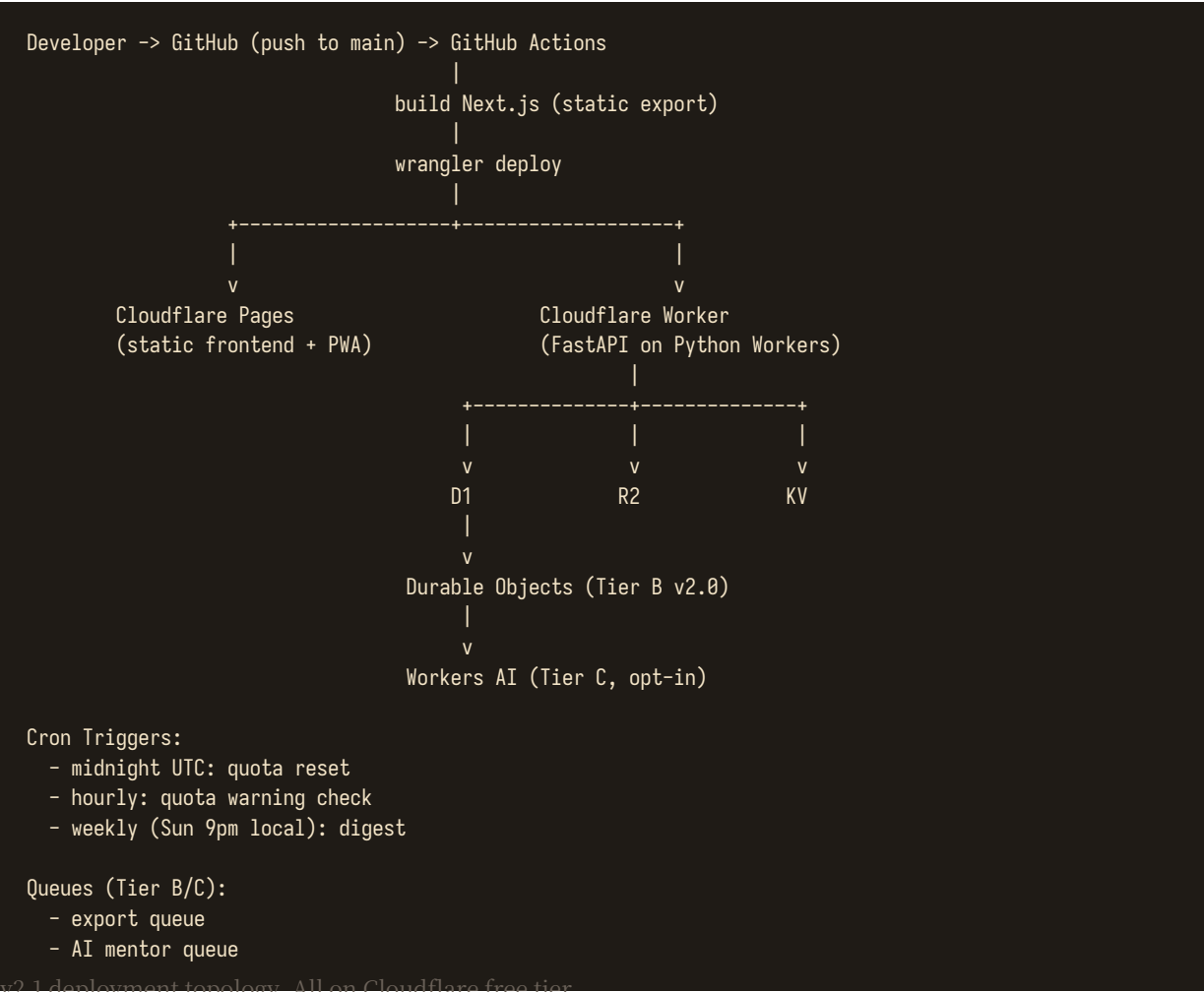
18. Observability

Signal	Mechanism	Key dimensions
Metrics	Cloudflare Workers Analytics (built-in, free) + custom quota dashboard in UI	Worker requests/day, D1 rows read/day, D1 rows written/day, R2 operations, KV operations, Workers AI neurons used, WebSocket connections, Queue operations.
Logs	Worker console.log -> Cloudflare Workers Logs (free tier: 24h retention)	request_id, route, status, duration, adapter-specific tags (AI quota hit/miss, KV cache hit/miss, D1 query count).
Traces	Cloudflare Workers Tracing (free)	UI -> Worker -> D1/R2/KV/DO/AI where useful.
Product analytics	D1 (learning_events table, partitioned by month)	lesson completion, quiz answer, simulation start/finish, challenge completion, review due/completed, note created, collection created, cost estimate generated, code walkthrough opened.
Quota dashboard	Custom widget in Progress page, polls /quota/today	Daily Worker/D1/R2/KV/AI usage with remaining quota and projected end-of-day usage.

19. Testing Strategy

- Unit tests for mastery calculations, scoring rules, scheduling, architecture validators, simulation engines (run in Node, not Pyodide).
- Contract tests for API schemas (Next.js client vs Worker server).
- Integration tests against D1 in local dev (Cloudflare D1 local emulator via wrangler).
- End-to-end tests for onboarding -> daily dose -> lesson -> quiz -> review. Run in Cloudflare Workers local emulator.
- Visual regression for core UI surfaces (light + dark mode, glass vs solid cards).
- Accessibility checks in CI (axe-core).
- Quota guard tests: simulate quota exhaustion, verify fail-closed behavior.
- Adapter fallback tests: simulate AI quota exhaustion, TTS unavailability, link preview timeout. Verify graceful degradation.
- Collab session tests (Tier B, v2.0): two-browser session, concurrent edits, snapshot save/load.
- No load tests needed (single user). Revisit if multi-user.

20. Deployment



v2.1 deployment topology. All on Cloudflare free tier.

Deployment is fully automated via GitHub Actions on push to main. No manual steps. No staging environment (single user, just deploy). Rollback via git revert + redeploy. D1 migrations run as a wrangler command in the deploy script.

21. Environments

Env	Purpose
Local	Feature development. wrangler dev for Worker + D1 local emulator. Next.js dev server. Seeded sample content from git.
Preview	Per-PR Cloudflare Pages preview deployment. Shareable URL for visual review.
Production	Single environment (single user). Deploy on push to main. Real Cloudflare free tier. Real D1/R2/KV.

No staging environment. Single-user personal app. Preview deployments on PRs are sufficient. Production is the only persistent environment. This eliminates an entire class of

staging/prod drift problems.

22. Migration Strategy

D1 uses SQLite migrations (forward-only SQL files in /migrations). wrangler d1 migrations apply runs them in order. Never mutate published content in place (content is in git). Backfill new mastery columns with explicit defaults. Keep migrations forward-safe and documented. v2.1 migrations add new tables incrementally per tier:

Versio n	Tables Added	Notes
0.1	users, review_items, learning_events, attempts, simulation_attempts, architecture_attempts, interview_sessions, achievements, user_achievements, quota_usage	Core Tier A tables. user_id is hardcoded = 1 in MVP (no multi-tenant complexity).
0.5	(none — 0.5 adds features using existing tables)	Case studies, projects, interview mode reuse existing attempt/session tables.
1.0	notes, annotations, highlights, collections, collection_items, cost_estimates, career_paths	Tier B tables. Forward-safe, all nullable defaults.
2.0	voice_sessions, voice_interview_attempts, collab_sessions, collab_participants, ai_mentor_sessions	Tier C/B tables. Forward-safe.

23. Technical Decisions to Avoid Early

- Managed PostgreSQL. Use D1. (Supabase/Neon free tiers have caveats like pause-on-inactivity.)
- Redis. Use KV + Durable Objects. No need for a separate cache layer.
- Celery/RQ worker pool. Use Queues + Cron Triggers.
- S3 + CloudFront. Use R2 (free egress is huge for a learning app with lots of images).
- Self-hosted Yjs server for collab. Use Durable Objects.
- Paid TTS providers (ElevenLabs, OpenAI TTS). Use Web Speech API.
- Paid STT providers. Use Web Speech API.
- Live cloud pricing APIs. Use bundled pricing dataset.
- Remote code execution (Firecracker, e2b, Judge0). Use Code Walkthrough Mode.
- Microservices. Stay modular monolith (single Worker).
- Kubernetes. Cloudflare handles scaling.
- Event streaming infrastructure. D1 partitioned tables are fine.
- Dedicated vector DB. Use D1 + cosine similarity in SQL if needed (small scale).
- Complex offline sync engine. PWA + lesson-pack downloads is enough.
- Realtime collaborative editing before core loop works. V2.0 feature.
- Custom auth stack. GitHub OAuth only.
- Multi-region deployment. Single-region (Cloudflare edge handles global latency).
- Mobile native shell (React Native) in MVP. PWA first; native is Tier D.
- Calendar integration in MVP. Tier C, opt-in.
- Multi-tenant transformation. Single user forever (unless app goes public, which is Tier D).

24. Zero-Cost Deployment Constraint (Non-Functional Requirement)

NFR: ZERO-COST DEPLOYMENT

No paid dependency and no accidental billing. Core functionality remains usable at free-tier exhaustion. Cloudflare's Free plan fails-closed (queries fail rather than silently switching to paid usage), which is the architectural property that makes this guarantee possible. When a free quota is reached: feature becomes unavailable, friendly fallback is shown, no bill is generated. Raw quota data lives under Settings → System Health (not prominent in the main UI).

24.1 Cloudflare Free Tier Allocations (as of 2026-08)

Service	Free Tier Allocation	NO CAP Expected Usage (single user)
Workers	100,000 requests/day	~5-15k/day (content is static; only mutations + user state reads)
D1	5 GB storage, 5M rows read/day, 100k rows written/day	Storage < 100MB; reads < 100k/day; writes < 5k/day
R2	10 GB storage, 1M Class A ops/month, 10M Class B ops/month, free egress	Storage < 1GB (diagrams + exports); ops minimal
KV	1 GB storage, 100k reads/day, 1k writes/day	Link preview cache only; < 1k reads/day
Durable Objects	100k requests/day, 13k GB-s/day	Only active during collab sessions (Tier B v2.0); < 1k/day
Queues	10k operations/day, 24h retention	< 100/day (review scheduling, digest)
Workers AI	10k neurons/day	< 2k/day (AI mentor is opt-in, Tier C)
Pages	Unlimited static hosting, 500 builds/month	1-5 builds/day (per-PR previews)
Cron Triggers	5 cron triggers per Worker (free)	3 in use (quota reset, quota warning, digest)

The free quotas are comfortably above expected single-user usage — not "effectively unlimited". A single user will not hit 100k Worker requests/day or 5M D1 rows/day in normal use, but the limits are real and the quota guards exist to enforce them gracefully. The tightest constraint is Workers AI (10k neurons/day) — hence the strict quota guard and deterministic fallback. Workers (100k requests/day) is mitigated by static-first content (zero Worker requests for content reads).

24.2 Quota Guard Pattern

Every optional service call (AI, link preview) is wrapped in a quota guard. The pattern:

```
function withQuotaGuard(service, requestFn, fallbackFn) {
  const usage = await getDailyUsage(service);
  const limit = FREE_TIER_LIMITS[service];

  if (usage >= limit * 0.95) {
    // Soft warning at 95%
    return fallbackFn('quota_near_limit');
  }

  try {
    const result = await requestFn();
    await incrementUsage(service, 1);
    return result;
  } catch (e) {
    if (e.code === 'QUOTA_EXHAUSTED') {
      return fallbackFn('quota_exhausted');
    }
    throw e;
  }
}

// Usage:
const mentorResponse = await withQuotaGuard(
  'workers_ai',
  () => aiMentor.generate(prompt),
  () => authoredFallback.explain(conceptSlug)
);
```

Quota guard pattern. Fail closed, never bill.

24.3 Quota Dashboard

The Quota Dashboard widget (visible in Progress page, Tier A) shows daily usage for each service. At 80%, a subtle banner appears: "Heads up: 80% of today's [service] quota used." At 95%, the relevant feature stops offering new requests and shows the fallback. At 100%, all requests fail-closed with "Quota exhausted for today - try again after midnight UTC." The learner is never asked to upgrade.

25. Architecture Evolution

v2.1 stays as a single Cloudflare Worker with clear module boundaries. The most likely future extractions (if usage grows or app goes multi-user) are: (1) collab service (Durable Objects already isolated), (2) AI service (already isolated behind quota guard). Both can be extracted to separate Workers without domain rewrite. The static-first content architecture means content can be moved to a separate repo + CDN without touching the app.

TRD DECISION

The strongest foundation for NO CAP is a Cloudflare-native, static-first, quota-guarded architecture around an ambitious learning model. Put complexity into content structure, simulations, evaluation, and mastery; keep the infrastructure simple and free. v2.1 adds optional services (AI, voice, collab) — each one earns its place by being wrapped in an adapter with a deterministic fallback and a hard quota guard. The result: a serious system-design workbench that actually lives at Rs 0/month forever.